```
import pandas as pd
import numpy as np

sample_mean = 68
population_mean = 65
n = 100

population_std = 4
z_score = (sample_mean - population_mean) / (population_std / np.sqrt(n))
print(f"The calculated Z-score is: {z_score}")
```

```
The calculated Z-score is: 7.5
```

```
from scipy import stats
import numpy as np

# Example data (replace with your actual data)
data = np.array([1.2, 2.5, 0.8, 1.5, 2.0, 0.9, 1.8, 2.1, 1.3, 1.7])

t_statistic, p_value_t = stats.ttest_1samp(data, popmean=1)
```

```
Start coding or generate with AI.
```

```
df=pd.read_csv('student_exam_performance_dataset.csv')
```

```
print(df.head())
```

```
  student_id  gender  age parental_education family_income internet_access  \
0     S00001    Male   17        High School        Medium             Yes
1     S00002  Female   18        High School           Low             Yes
2     S00003    Male   17        High School        Medium              No
3     S00004    Male   18           Bachelor        Medium             Yes
4     S00005    Male   18           Bachelor        Medium             Yes

  study_environment  study_hours_per_day  attendance_rate  sleep_hours  ...  \
0             Quiet                 2.98             96.5         6.05  ...
1             Quiet                 4.45             95.7         6.96  ...
2             Quiet                 3.75             76.0         7.02  ...
3             Quiet                 2.03             72.6         6.23  ...
4             Quiet                 5.14             87.3         8.54  ...

   online_courses_completed  tutoring  math_score  reading_score  \
0                         1       Yes        42.8           62.4
1                         0       Yes        77.9           73.5
2                         4       Yes        53.5           38.3
3                         4        No        28.3           23.5
4                         0        No        74.7           54.9

  writing_score  science_score  final_exam_score  previous_gpa  pass_fail  \
0          54.8           51.8              49.1          2.44       Fail
1          64.4           61.6              70.1          2.79       Pass
2          36.3           47.1              42.2          1.49       Fail
3          32.0           39.0              31.9          1.34       Fail
4          73.6           55.5              66.4          2.60       Pass

   grade_category
0               F
1               C
2               F
3               F
4               C

[5 rows x 23 columns]
```

```
n=len(df)
```

```
sample_mean = df['final_exam_score'].mean()
```

```
sample_std = df['final_exam_score'].std(ddof=1)
```

```
confidence = 0.95
t_critical=stats.t.ppf(
    (1+confidence)/2,
    df=n-1
)
```

```
from scipy import stats
import matplotlib.pyplot as plt
```

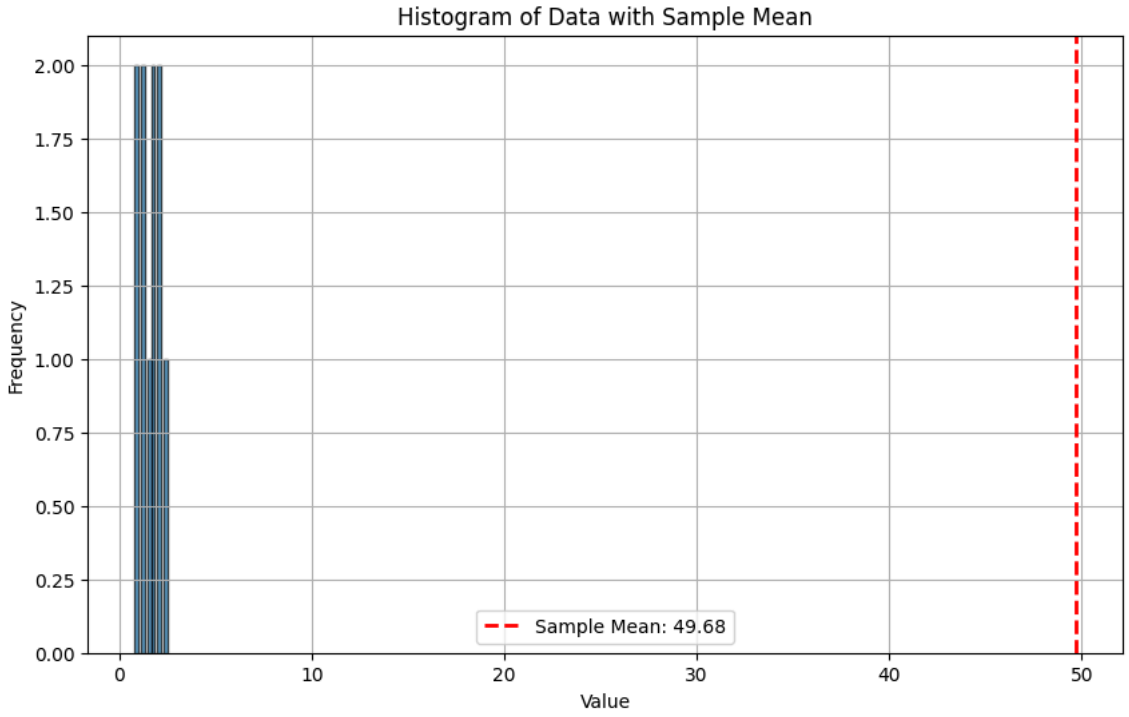
```
plt.figure(figsize=(10, 6))

plt.hist(data, bins=10, edgecolor='black', alpha=0.7)

plt.axvline(
    sample_mean,
    color='red',
    linestyle='dashed',
    linewidth=2,
    label=f'Sample Mean: {sample_mean:.2f}'

)

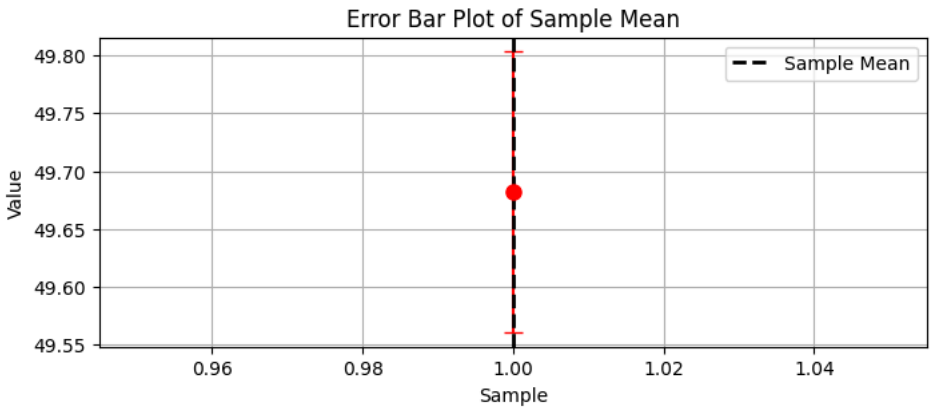
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.title('Histogram of Data with Sample Mean')
plt.legend()
plt.grid(True)
plt.show()
```




```
plt.figure(figsize=(8,3))
plt.errorbar(
    x=[1],
    y=[sample_mean],
    yerr=[sample_std/np.sqrt(n)],
    fmt='o',
    color='red',
    capsize=5,
    markersize=8
)

plt.axvline(
    x=1,
    color='black',
    linestyle='dashed',
    linewidth=2,
    label='Sample Mean'
)

plt.xlabel('Sample')
plt.ylabel('Value')
plt.title('Error Bar Plot of Sample Mean')
plt.legend()
plt.grid(True)
plt.show()
```




```
mu=70
population_std=10
```

```
z_score=(sample_mean-mu)/(population_std/np.sqrt(n))
```

```
p_value_z=2*(1-stats.norm.cdf(np.abs(z_score)))
print("Hypothesized Mean:",mu)
print("Sample Mean:",sample_mean)
print("Sample Size:",n)
print("Z-Score:",z_score)
print("P-Value:",p_value_z)
```

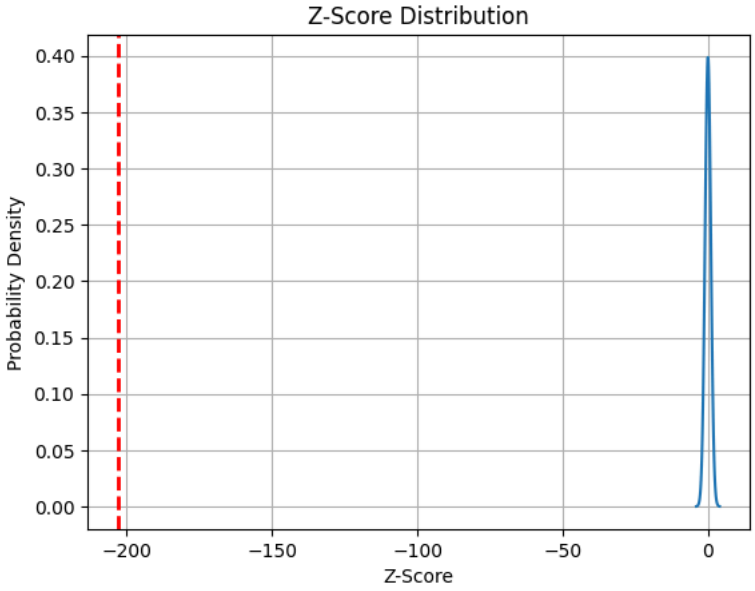
```
Hypothesized Mean: 70
Sample Mean: 49.681909999999995
Sample Size: 10000
Z-Score: -203.18090000000004
P-Value: 0.0
```

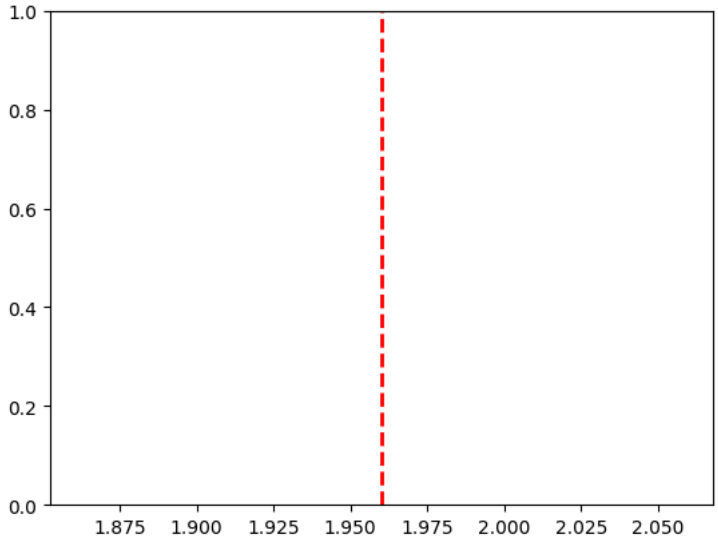
```
alpha=0.05
if p_value_z<alpha:
    print("Reject the null hypothesis")
else:
    print("Fail to reject the null hypothesis")
```

```
Reject the null hypothesis
```

```
x=np.linspace(-4,4,100)
y=stats.norm.pdf(x)
plt.plot(x,y)
plt.axvline(z_score,color='red',linestyle='dashed',linewidth=2)
plt.xlabel('Z-Score')
plt.ylabel('Probability Density')
plt.title('Z-Score Distribution')
plt.grid(True)
plt.show()

z_critical=1.96
plt.axvline(z_critical,color='red',linestyle='dashed',linewidth=2)
plt.show()
```

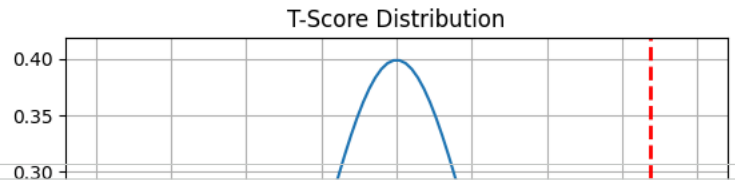





```
t_statistic, p_value_t = stats.ttest_1samp(data, popmean=1)
print("T-Statistic:", t_statistic)
print("P-Value:", p_value_t)
```

```
T-Statistic: 3.3762547868225488
P-Value: 0.008175414613500787
```

```
x_t=np.linspace(-4,4,100)
y_t=stats.t.pdf(x_t,df=n-1)
plt.plot(x_t,y_t)
plt.axvline(t_statistic,color='red',linestyle='dashed',linewidth=2)
plt.xlabel('T-Score')
plt.ylabel('Probability Density')
plt.title('T-Score Distribution')
plt.grid(True)
plt.show()
```




```
plt.figure(figsize=(7,5))
values=[mu,sample_mean]
labels=['Hypothesized Mean','Sample Mean']
plt.bar(labels,values,color=['blue','red'])
plt.xlabel('Value')
plt.ylabel('Mean')
plt.title('Hypothesized Mean vs Sample Mean')
plt.grid(True)
plt.show()
```

